Cursor Case Study

Michael Incorvaia and Greyson McReynolds and Heeba Merchant and Canting Zhu
ECE 6256 / CS 6604 - Prof. Heck

Abstract

Cursor is an AI code editor that wraps frontier models from several vendors, along with its own models, in an agent harness that searches, edits and runs a developer's repository. We map Cursor onto the classical decomposition of understanding, state, policy, generation and their interfaces, using Cursor's engineering posts, third-party reporting and our own runs. We find that understanding and state are partly explicit, built from indexing, retrieval and file-based context, while generation stays latent in the model. Policy is latent but increasingly owned by Cursor through reinforcement learning in its own harness, and interfaces are explicit but written separately for each vendor. The seams appear where the harness ends: in pricing, in switching models mid-chat, and in a support bot with no grounding. In two runs of a fixed coding task, both agents passed all nine held-out tests, but one misestimated its own run time by a factor of 2.65, showing that an agent's account of itself needs an outside check.

1 Cursor Introduction

Cursor is an AI code editor forked from VS Code. Developed by Anysphere and first released in 2023, it has changed how many developers write software. It wraps frontier models (Anthropic, OpenAI, Google, xAI) and its own models (Tab; Composer 2, built on Moonshot's Kimi K2.5) in an agent harness that reads, searches, edits and runs a developer's repo. The user is a developer; the "world" the dialogue acts on is the codebase and terminal.

Rather than focus only on how tools like this change engineering practice, we survey the decomposition of the system itself: which of the five classical boxes (understanding, state, policy, generation, and their interfaces) Cursor makes explicit, and which it leaves latent in the model. We also probe the places where Cursor's design breaks down, from its pricing and model switching to a support bot that invented a company policy, as we believe they show where the explicit harness ends and the latent model takes over.

Box	Status	In Cursor
Understanding	Both	Indexing and semantic search ground the request; the model interprets it
State	Both	Static context plus files for history and tool output; the model chooses what to fetch
Policy	Latent	Chosen in the model; shaped by Tab's reward, Composer's RL, plan mode and Router
Generation	Latent	Model output, reviewed by the user as diffs
Interfaces	Explicit	Per-vendor edit formats and prompts; typed tool errors
Missing	None	Grounding outside the repo; a measure of the user's actual goal

Table 1: Cursor mapped onto the classical decomposition.

2 Classical Boxes

For each of the five boxes, we examine how Cursor implements it and whether that implementation is explicit, latent in the model, or missing. Table 1 summarizes our answers; the subsections below give the evidence.

2.1 Understanding

Even prior to a user submitting a prompt, Cursor has already performed work towards 'understanding' via an indexing pipeline. Initially, this pipeline divides the codebase into syntactic chunks which are then converted into embeddings. When a user modifies files, Cursor uses a two-part system consisting of Merkle trees and caching to prevent inefficient recomputation. Merkle trees, hierarchical data structures that use cryptographic hashes to represent files and directories, allow Cursor to efficiently identify which files have changed. From

this, the altered chunks can be determined. Then, Cursor reuses the cached embeddings for the unchanged chunks and only generates new embeddings for those that were modified (Cursor, 2026b).

When a user submits a request, Cursor is able to 'understand' the user by searching its indexed codebase for semantically related code (Heule et al., 2025). The request and the retrieved code are then passed to the language model, which decides how to reason about the request in that context; Cursor's own Composer is a mixture-of-experts model (Cursor, 2025b).

We would consider this system to be both explicitly defined as well as latent in the model. It is explicitly defined in the way that the indexing pipeline is intentionally set up for the purpose of understanding the codebase, and thereby, the user's requests. However, the language model's interpretation of the request is learned, and therefore latent.

2.2 State

Cursor pulls context from many sources. At its most basic level, it pulls static context such as the operating system, git status, and current and recently viewed files (Heule and Katz, 2026). However, much of Cursor's context is dynamically sourced (and dynamically managed). For example, long conversation histories are written into files that Cursor can re-read when needed. This is decided on by the underlying LLM, but performed by the agent harness; the harness has the ability to assemble relevant states into the language model's context window (Katz, 2026). Additionally, Cursor has the ability to trigger a self-summarization step when the model is approaching its context limit, allowing it to compress its understanding into a smaller capacity. This is especially useful when working on longer tasks (Cursor, 2026a).

We would again consider state both explicit and latent: it is explicitly stored and maintained by procedures such as self-summarization, while the model learns what is relevant, when to retrieve more, and what to keep when it summarizes.

2.3 Policy

In the classical decomposition, the policy maps the belief state to the next dialogue act, such as asking, confirming, or acting, and it is learned against a reward that makes the trade-off between asking and acting explicit. In Cursor, the next act is usually a tool call: search the codebase, read a file, make an edit, run a terminal command, ask the user a question, or stop and answer. That choice is made token by token inside whichever model is running, so at its core the policy is latent. However, Cursor does not leave it alone; it shapes the policy at three levels.

The clearest example is Tab, Cursor's autocomplete model. Its policy has two actions, suggest or stay silent, and its reward is written down: $+0.75$ for an accepted suggestion, -0.25 for a rejected one, and 0 for silence, so a suggestion only pays when its chance of acceptance is above 25%. Trained on-policy on over 400 million requests per day, Tab learned to make 21% fewer suggestions with a 28% higher accept rate (Jackson et al., 2025).

For the agent, Cursor trains its own models inside its own harness. Composer was trained with reinforcement learning on the same tools the product exposes, in sandboxed coding environments, and was pushed to make efficient tool choices, run tools in parallel, and avoid claims made without evidence (Cursor, 2025b). Composer 2 also learns its own context summaries. The final task reward is applied to the whole chain, summaries included, so a summary that drops critical information is penalized; this cut compaction error by 50% compared with a tuned prompt-based baseline while using one fifth of the tokens (Cassano and Rush, 2026). In effect, the policy has learned to manage its own state.

For the frontier models Cursor rents, it cannot change the weights, so it adds explicit controls instead. Plan mode has the agent research the codebase, ask clarifying questions, and write an editable plan before changing anything (Smith, 2025), which works as a user-selectable "confirm before acting" policy. Cursor Router, the relaunched Auto mode, is a classifier that picks a model for each request based on its task type and complexity, trained on over 600,000 live requests with user satisfaction as the reward (Cursor Team, 2026). Meanwhile, earlier guardrails such as a cap on tool calls per turn have mostly been removed as the models improved (Heule and Katz, 2026).

Our own Grok 4.6 High Fast run shows this policy at work (Table 2). We recovered the full conversation from Cursor's local chat history: 24 tool calls, or 19 without its four to-do updates and one tool lookup. Before editing, the agent searched for the relevant symbols and then read six files in a single parallel batch, the same behavior Cursor trains Composer toward. After editing, it wrote tests of its own beyond the six it was given; its

Calls	Box	What the agent did
2–3	U	Globbed the repo and searched for `class Library`, `fees_owed` and `def report`
4	S	Wrote a to-do plan
5–10	S/P	Read six files in one parallel batch
14–16	G	Edited `library.py` (twice) and `cli.py`
17	G	Wrote its own tests for the premium grace period, the $10 cap, still-out loans and the report line
19	I	`pytest` blocked: "Terminal unavailable" sandbox error ($t = 48$ s)
22	P	Reran outside the sandbox: 10 passed ($t = 50$ s)
23	P	Ran the CLI on sample data to check the report ($t = 64$ s), then stopped

Table 2: Annotated excerpt of our Grok 4.6 High Fast run, recovered from Cursor's local chat history. Calls number all 24 tool calls in order. U, S, P, G and I mark understanding, state, policy, generation and interfaces.

reasoning summary states that "a hidden test suite likely exists." It then ran the suite and checked the CLI output on the sample data before stopping.

We would consider policy to be latent, but increasingly owned by Cursor. The next action is chosen inside the model, but Cursor shapes that choice with a hand-written reward (Tab), reinforcement learning in its own environment (Composer), and a learned proxy for user satisfaction (Router). None of these rewards observe whether the user's actual goal was met.

2.4 Generation

Generation is the most latent of the five boxes. The code and text that Cursor produces come straight from the underlying model, whether that is a rented frontier model or Composer. What Cursor controls is how that output reaches the codebase. Edits are shown to the developer as diffs to review and accept or reject, and the agent can run tests and read their output before it finishes, as it did in both of our runs (Table 2). We would therefore consider generation latent, with an explicit review step placed between the model's output and the user's code.

2.5 Interfaces

The interfaces between the boxes are the most explicit part of Cursor, and they are written separately for each model vendor. The clearest example is the edit format. OpenAI models are given a patch-based format, while Anthropic models are given string replacement, because each model was trained on its own format and an unfamiliar one "costs extra reasoning tokens and produces more mistakes" (Heule and Katz, 2026). Prompts are also tuned per provider and even per model version (Heule and Katz, 2026).

The interface from the environment back to the policy is explicit as well. When a tool call fails, the harness returns a typed error, such as `InvalidArguments`, `UnexpectedEnvironment`, `ProviderError`, `UserAborted`, or `Timeout`, and any other error is treated as a bug in the harness itself (Heule and Katz, 2026). This plays the role of the observation in the classical model, since it tells the policy what kind of failure happened rather than only that one did. Our Grok run hit this channel directly. Its first `pytest` call came back as a "Terminal unavailable" error, because the machine could not enforce the workspace sandbox the command required, and the message stated that this was "not a permission denial." The agent read the error, reran the tests outside the sandbox two seconds later, and all ten passed (Table 2).

Because these interfaces are built per vendor, they are not a universal schema in the way a dialogue-act ontology would be. Instead, they form a set of one-to-one adapters, one per model, and this is the source of the model-switching seam discussed below. We would consider interfaces explicit, and more carefully engineered than any other box.

3 The Seam

Cursor's weak points show up in two places. The first is where its harness meets a model it doesn't own, and the second is where a conversation happens outside the codebase. Looking at four documented problems, each one points back to a specific box in the decomposition.

The first problem was cost. Until mid-2025, Cursor Pro gave users 500 requests a month. In agent mode, one request can set off a long chain of tool calls, so some requests cost far more than others, while Cursor pays model vendors by the token. In June 2025, Cursor replaced the request limit with $20 of frontier-model usage per month at API prices, plus unlimited use of Tab and Auto mode (Cursor, 2025a). Many users still expected the old 500-request limit and got surprise bills. Cursor apologized that the change "was not communicated clearly" and refunded unexpected charges from June 16 to July 4 (Cursor, 2025a). This shows a gap between policy and cost. The agent decides

how many tokens a task uses, but Cursor doesn't control what those tokens cost. It also explains why Cursor works so hard on context management. Dynamic context discovery cut token use nearly in half on MCP runs (Katz, 2026), which improves quality and protects Cursor's margins at the same time.

The second problem appears when users switch between models. Because Cursor gives each model its own tool formats and prompts, changing models in the middle of a chat is harder than it looks. The new model has to read a history written by a different model, it may try to call tools it doesn't have, and the prompt cache resets, which makes the next turn slower and more expensive (Heule and Katz, 2026). Cursor's own advice is to stick with one model per chat, or to start a subagent with a fresh context (Heule and Katz, 2026). So the decomposition isn't built once. The boxes stay the same, but Cursor rebuilds the connections between them for every model.

To get out from under both of these problems, Cursor started training its own model. Composer 2 launched on March 19, 2026. It costs about 90% less per token than Claude Opus 4.6 and scores higher on two coding benchmarks (DataCamp, 2026). However, the base model wasn't Cursor's. A day after launch, developers found a model ID in API requests showing it was built on Moonshot AI's open Kimi K2.5 (DataCamp, 2026). Cursor said about three quarters of the training compute was its own, called the lack of disclosure "a miss," and promised to name base models in the future (DataCamp, 2026). This shows Cursor bringing the model in-house, but only partly. It now shapes policy by training with reinforcement learning in its own harness (Cursor, 2025b), while understanding and generation still depend on another company's pretrained model.

The clearest failure, though, happened outside the editor entirely. In April 2025, a bug started logging users out when they switched devices. Cursor's AI support bot, which signed its emails as "Sam," told customers this was expected under a new one-device policy. That policy didn't exist, and users publicly canceled their subscriptions (Fortune, 2025). Co-founder Michael Truell later apologized for "an incorrect response from a front-line AI support bot." Unlike the editor, the support bot couldn't look up actual company policy, had no information about the outage, and nothing checked its answer before it went out. If you only count whether the bot handled the ticket without a human, it succeeded. If you count whether it actually solved the customer's problem, it failed, because the customer left. Cursor built careful grounding into its product but left it out of everything else.

4 Measured Behavior

We evaluated Cursor on a fixed task: adding late fees to a small Python library application while preserving existing behavior. The requirements included a daily fee, a per-loan cap, a premium grace period, and a fee line in the CLI report. We planned a four-model comparison, but the available Free account rejected named-model requests, so we report our Grok 4.6 High Fast editor run and one Auto CLI run. The Grok run took 74 seconds and 19 tool calls (Table 2); its saved patch passes all nine held-out tests. The Auto coding invocation took 33.9 seconds, made 14 tool calls, and passed all six original visible tests and all nine held-out tests. In a separate follow-up after timing stopped, Auto estimated its duration as 90 seconds, approximately 2.65 times the measured duration, while correctly estimating 14 tool calls. These observations do not support a model ranking: the runs used different interfaces, Auto did not disclose its underlying model, each configuration ran once, token totals used different reporting conventions, and dollar costs were unavailable. Held-out test success also measures behavior on those cases, not long-term code retention.

5 Verdict

Cursor combines explicit infrastructure with latent model behavior. Search, file access, editing, and execution tools connect the model to an observable codebase, while the model decides which actions to take through them, as our Grok trace shows (Table 2). Held-out tests confirmed both runs' changes, but a passing patch does not show that the system understood the user's broader goal, and the inaccurate duration estimate shows that an agent's account of itself needs an outside check. Our measurements support only a narrow conclusion, that Cursor completed this bounded task; the broader case for context management and interface engineering rests on the public evidence above.

References

Federico Cassano and Sasha Rush. 2026. Training Composer for longer horizons. Cursor Blog.

Cursor. 2025a. Clarifying our pricing. Cursor Blog.

Cursor. 2025b. Composer: Building a fast frontier model with RL. Cursor Blog.

Cursor. 2026a. Introducing composer 1.5. Cursor Blog.

Cursor. 2026b. Securely indexing large codebases. Cursor Blog.

Cursor Team. 2026. Introducing Cursor Router. Cursor Blog.

DataCamp. 2026. Composer 2. DataCamp Blog.

Fortune. 2025. Cursor's customer support AI went rogue. Fortune.

Stefan Heule, Emily Jia, and Naman Jain. 2025. Improving agent with semantic search. Cursor Blog.

Stefan Heule and Jediah Katz. 2026. Continually improving our agent harness. Cursor Blog.

J. Jackson, P. Kravtsov, and S. Jain. 2025. Improving Cursor Tab with online RL. Cursor Blog.

Jediah Katz. 2026. Dynamic context discovery. Cursor Blog.

Jai Smith. 2025. Introducing Plan Mode. Cursor Blog.